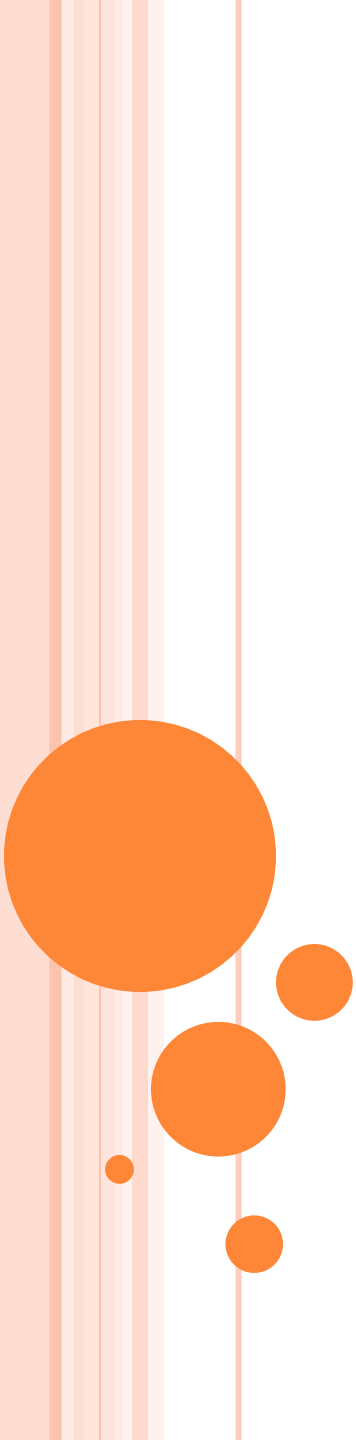




PROGRAMOWANIE GIER W UNITY I C#

WYKŁAD 5 – MODELOWANIE I GENEROWANIE TERENU, POMIJANIE OBIEKTÓW PRZESŁONIĘTYCH, ANIMACJE – DRZEWA MIESZAJĄCE, RAGDOLL, OBIEKTY SKRYPTOWE

Bogumiła Hnatkowska

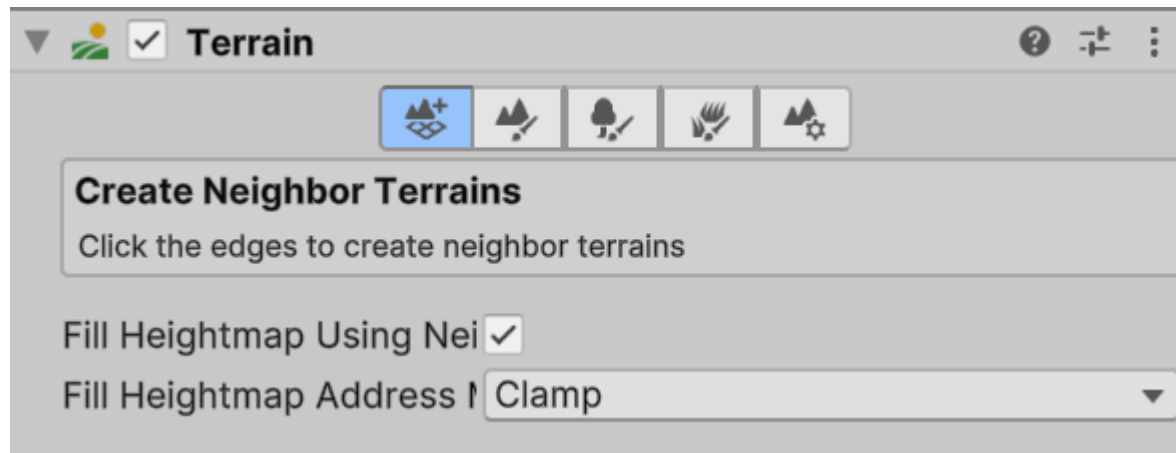
MODELOWANIE TERENU – PODSTAWOWE POJĘCIA

- Teren (Terrain) – komponent Unity reprezentujący siatkę terenu
- Mapa wysokości (Heightmap) – tablica, przechowująca informację o wysokości terenu w skali 0-1 dla każdego punktu terenu
- Poziom terenu (Terrain Layers) – warstwa materiałów (tekstura), nakładana na teren; na teren może być nałożonych wiele poziomów (wbudowany potok renderowania obsługuje do 4)
- Splatmapa (Splatmap, control texture) – tekstura RGBA, gdzie każdy kanał odpowiada za intensywność jednej warstwy tekstury terenu dla danego punktu; opisuje sposób mieszania tekstur (trawa, ziemia) na powierzchni siatki terenu
- Kafelek terenu – niezależny obiekt terenu, który jest częścią większej całości



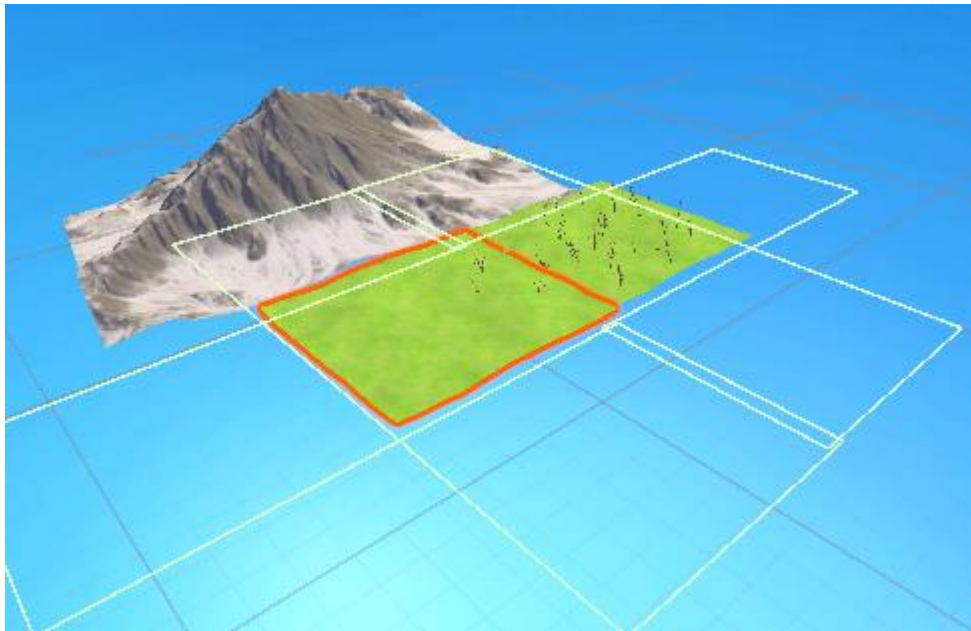
MODELOWANIE TERENU, C.D.

- GameObject > 3D Object > Terrain
- Inspektor terenu dostarcza narzędzi do jego definicji
 - Tworzenie sąsiadujących kafelków terenu
 - Rzeźbienie i malowanie terenu
 - Dodawanie drzew
 - Dodawanie szczegółów (trawa, kwiaty itp.)
 - Zmiana ustawień



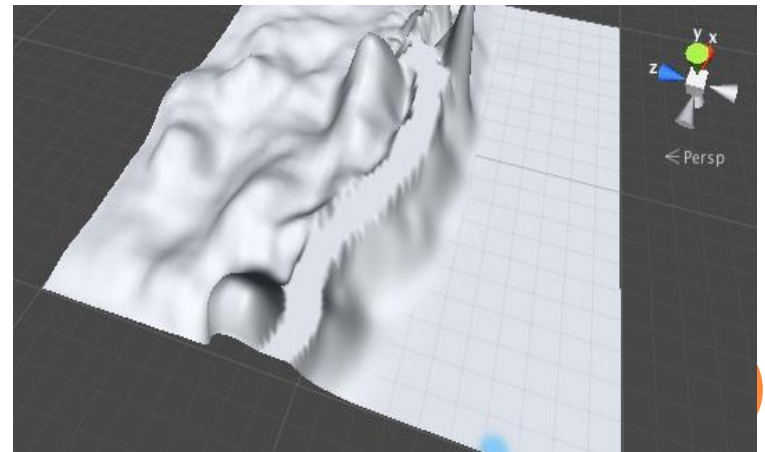
MODELOWANIE TERENU, C.D.

- Tworzenie sąsiadujących kafelków terenu



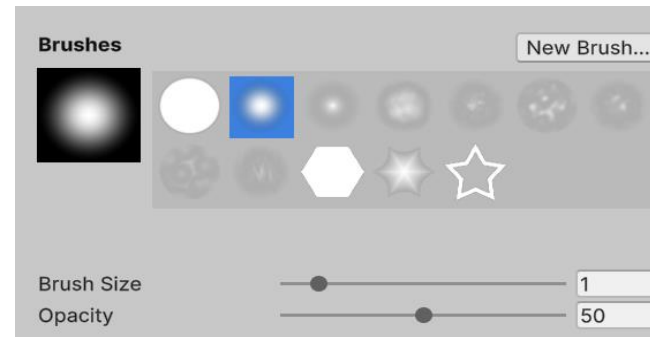
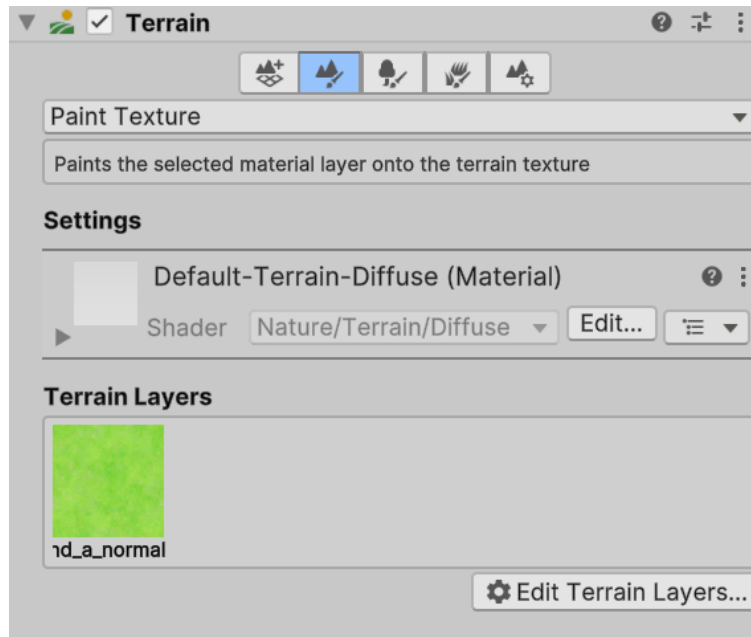
MODELOWANIE TERENU

- Rzeźbienie i malowanie terenu: 
 - Podnoszenie i opuszczanie terenu – budowa mapy wysokości
 - Malowanie dziur (wgłębień)
 - Malowanie teksturami
 - Ustawianie wysokości
 - Wygładzanie wysokości
 - Stemplowanie terenu



MODELOWANIE TERENU, C.D.

- Tworzenie warstwy terenu:

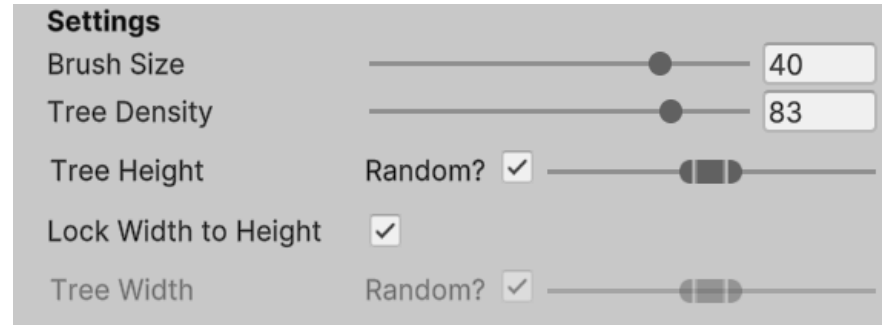
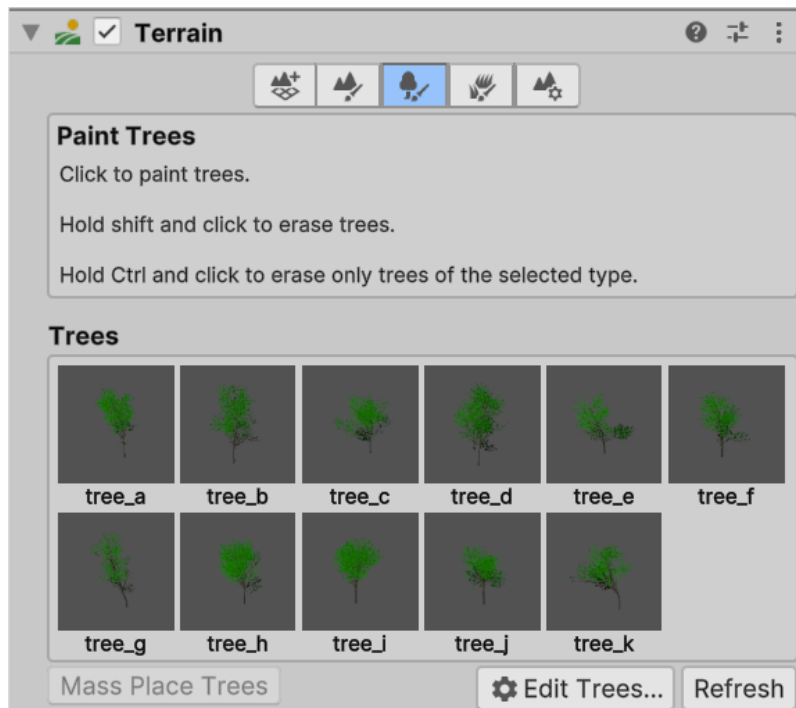


- Przykładowe tekstury terenu, drzewa itp. są dostępne np. w pakiecie Free Stylized Nature Environment (sklep)



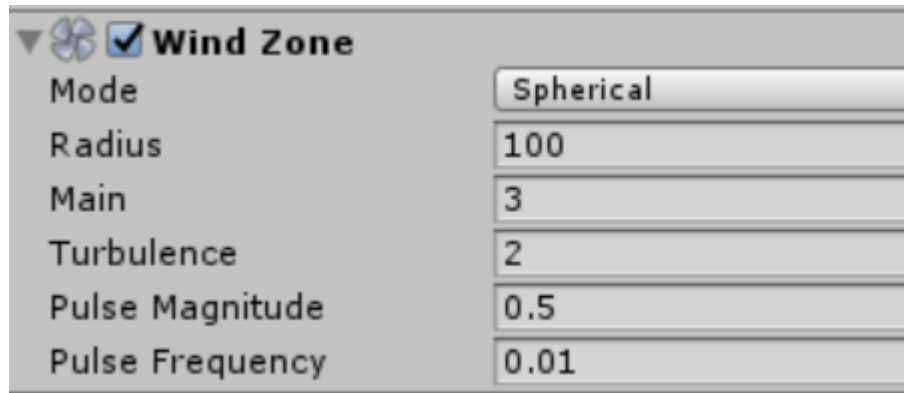
MODELOWANIE TERENU, C.D.

- Dodawanie drzew: 



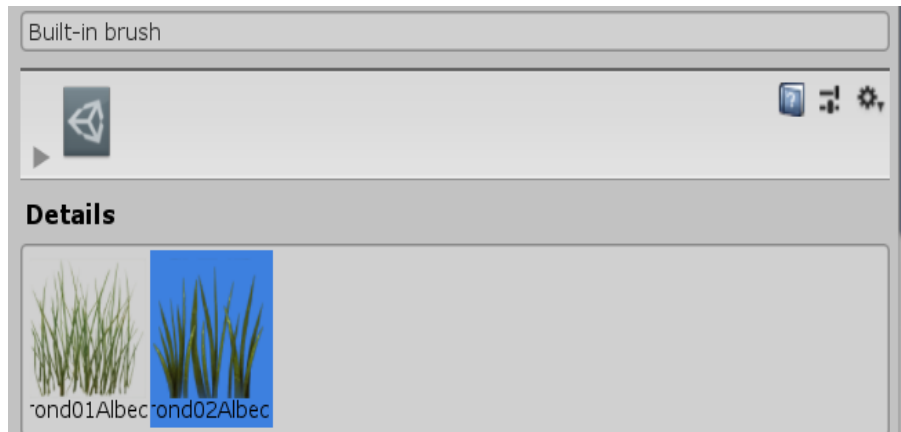
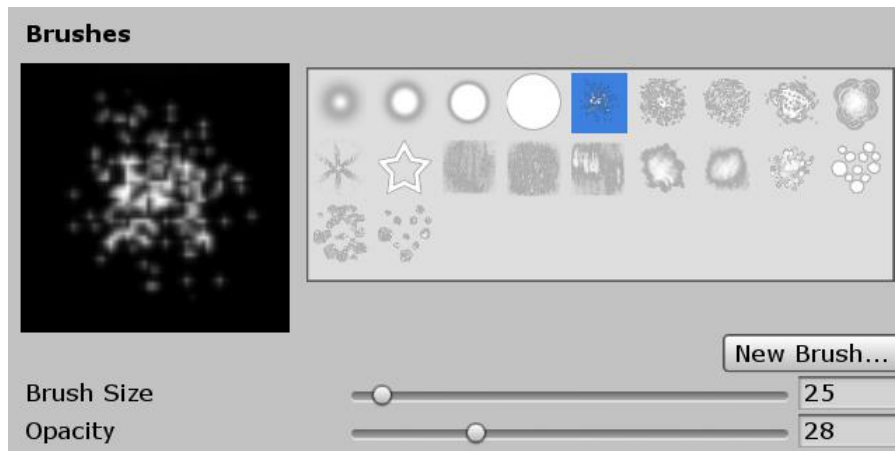
MODELOWANIE TERENU, C.D.

- Dodawanie stref wiatru:
 - Dodaj komponent: Wind Zone



MODELOWANIE TERENU, C.D.

○ Dodawanie szczegółów



MODELOWANIE TERENU, C.D.

○ Wybrane ustawienia terenu

▼ **Mesh Resolution (On Terrain Data)**

Terrain Width	75
Terrain Length	75
Terrain Height	10
Detail Resolution Per	8
Detail Resolution	1024

▼ **Texture Resolutions (On Terrain Data)**

 Require resampling on change

Heightmap Resolution	1025 x 1025	▼
Import Raw... Export Raw...		
Control Texture Resol	512 x 512	▼
Base Texture Resoluti	1024 x 1024	▼

Rozdzielczość splatmapy



GENEROWANIE TERENU

- Moment generowania:
 - Online
 - Offline
- Konieczność istnienia obiektów:
 - Generowana zawartość niezbędna do ukończenia gry
 - Generowana zawartość dodatkowa, nie mająca wpływu na przebieg gry
- Liczba prób generacji:
 - Konstruktywny (w szczególności: generuj i popraw)
 - Generuj i testuj
- Symulacja procesów terenotwórczych:
 - Teleologiczny – symuluje procesy, mające wpływ na kształtowanie środowiska, np. pożar
 - Ontogenetyczny – nie symuluje procesów, najczęściej oparty o szumy

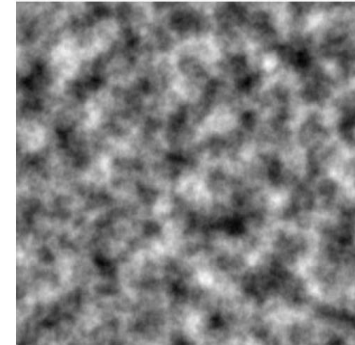


PRZYKŁADOWE SZUMY

- Szum Perlina

Dostępny od Unity 2020.3

`Mathf.PerlinNoise`

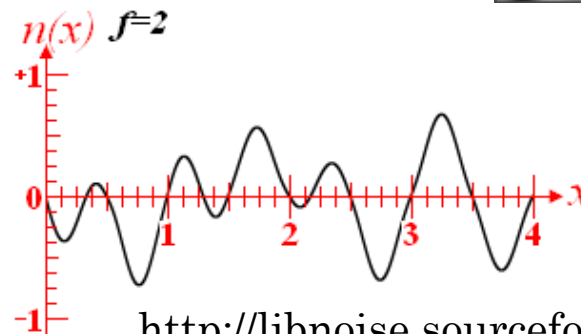
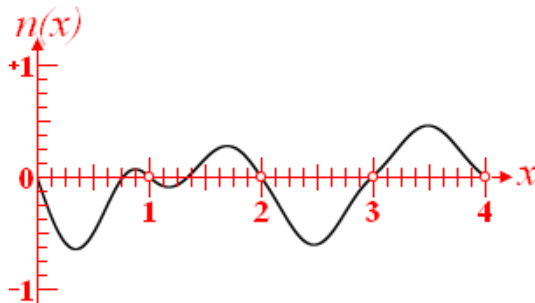
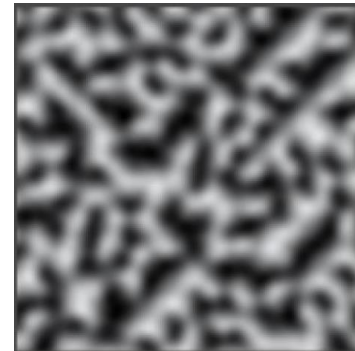


- Szum simpleksowy

Przykładowa implementacja:

[https://github.com/](https://github.com/Auburn/FastNoise_CSharp)

`Auburn/FastNoise_CSharp`



<http://libnoise.sourceforge.net/glossary/index.html>



GENERACJA BIOMÓW

- Biom – obszar o określonym klimacie, charakterystycznej szacie roślinnej i świecie zwierzęcym
- Przykłady (za wikipedią):
 - Pustynie lodowe
 - Tundra
 - Tajga
 - Stepy
 - Pustynie
 - Lasy liściaste
 - ...



GENERACJA BIOMÓW, C.D.

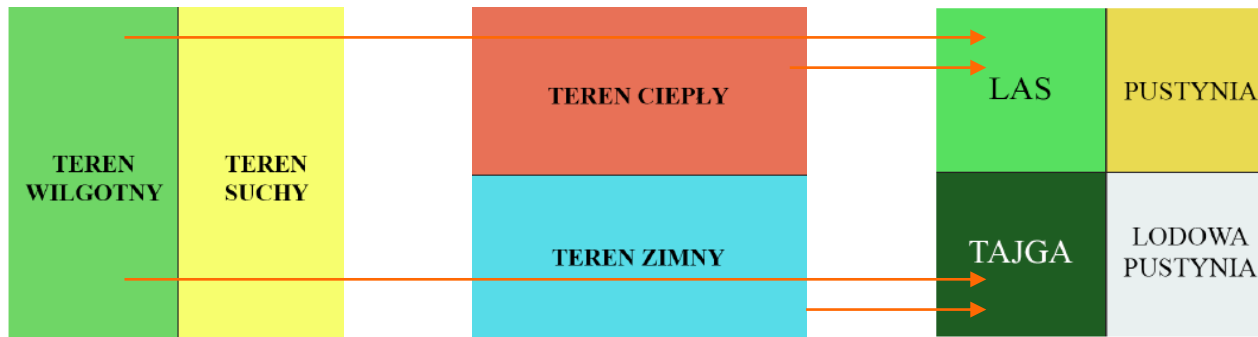
- Biomy mogą być generowane na podstawie map (ich dowolnej kombinacji):
 - Wilgotności
 - Temperatury
 - Wysokości
- Każda z map może być wygenerowana z wykorzystaniem szumu

The image shows a vertical menu of settings for terrain generation, organized into six sections. Each section has a title, several input fields, and a button at the bottom.

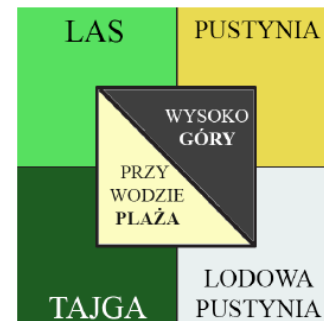
- Ustawienia mapy wysokości**
 - Ziarno: 21949489
 - Skala: 4
 - Maks. wysokości: 3.5
 - Rodzaj szumu: Perlin
- Ustawienia wody**
 - Wysokość wody: 12
 - GENERUJ TEREN
- Ustawienia mapy wilgotności**
 - Rodzaj szumu: Perlin
 - Skala: 4
 - Ziarno: 21949489
 - WIZUALIZUJ MAPĘ WILGOTNOŚCI
- Ustawienia mapy temperatury**
 - Rodzaj szumu: Perlin
 - Skala: 4
 - Ziarno: 21949489
 - WIZUALIZUJ MAPĘ TEMPERATURY
- Ustawienia biomów**
 - Wysokość plaży: 1.5
 - Wysokość gór: 12.7
 - Wygładzenie: Wysokie
 - GENERUJ BIOMY
- Ustawienia obiektów**
 - Liczba typów drzew: 10
 - Gęstość drzew: 21.7
 - Gęstość detali: 100
 - GENERUJ OBIEKTY

GENERACJA BIOMÓW, C.D.

- Przykład generacji biomów na podstawie map wilgotności i temperatury (wartości progowe, kiedy teren wilgotny/suchy ...)



- I dodatkowo, wysokości



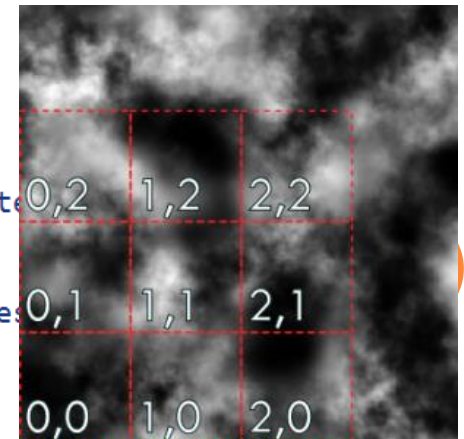
PRZYKŁAD GENERACJI TERENU

```
public class TerrainChunk
{
    Odwołania: 5
    private Vector3 Position { get; set; }
    Odwołania: 2
    private Terrain Terrain { get; set; }           // wygenerowany teren
    Odwołania: 5
    private int TerrainWith { get; set; }           // rozmiar terenu w pikselach
    Odwołania: 2
    private int TerrainHeight { get; set; }         // maksymalna wysokość
    Odwołania: 9
    private int HeightMapResolution { get; set; }  // rozdzielczość mapy wysokości
    Odwołania: 2
    private INoiseProvider NoiseProvider { get; set; } // generator wysokości

    1 odwołanie
    public TerrainChunk(int terrainWith, int terrainHeight, INoiseProvider noiseProvider,
        int x, int z) {...}
    1 odwołanie
    public void CreateTerrain(Texture2D[] textures){...}
    1 odwołanie
    private TerrainData PrepareTerrainData() {...}

    1 odwołanie
    private void CreateTerrainLayers(TerrainData terrainData, Texture2D[] textures) {...}

    1 odwołanie
    private void ApplyTextures(TerrainData terrainData, Texture2D[] textures) {...}
    1 odwołanie
    private float[,] GetHeightmap() {...}
}
```



PRZYKŁAD GENERACJI TERENU, C.D.

○ Generacja mapy wysokości

```
private float[,] GetHeightmap()  
{  
    var heightmap = new float[HeightMapResolution, HeightMapResolution];  
  
    for (var zRes = 0; zRes < HeightMapResolution; zRes++)  
    {  
        for (var xRes = 0; xRes < HeightMapResolution; xRes++)  
        {  
            var xCoordinate = Position.x + (float)xRes / (HeightMapResolution - 1);  
            var zCoordinate = Position.z + (float)zRes / (HeightMapResolution - 1);  
  
            heightmap[zRes, xRes] = NoiseProvider.GetValue(xCoordinate, zCoordinate);  
        }  
    }  
    return heightmap;  
}
```



PRZYKŁAD GENERACJI TERENU, C.D.

```
public interface INoiseProvider
{
    float GetValue(float x, float z);
}

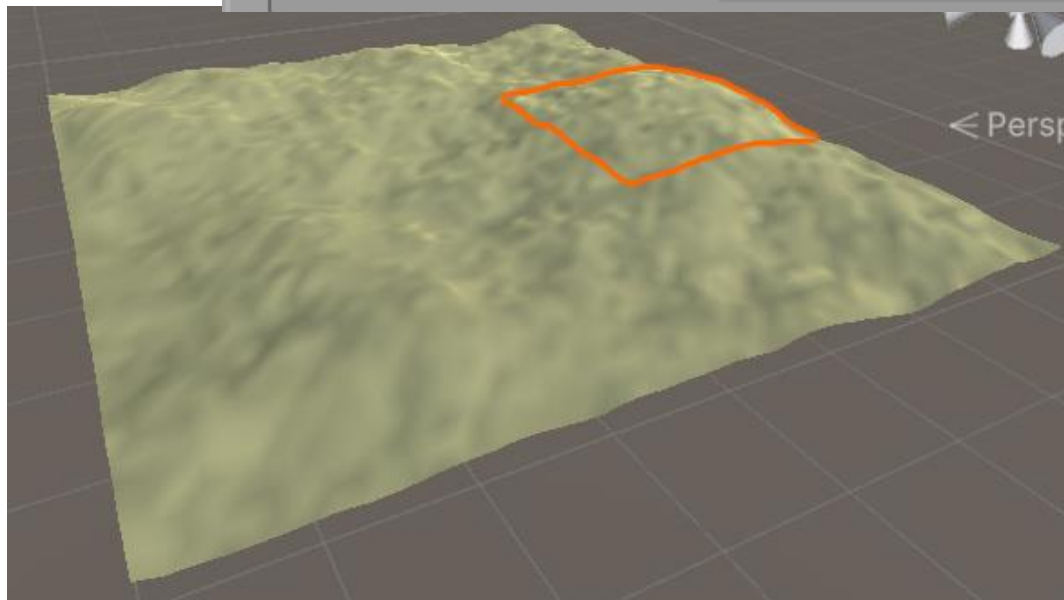
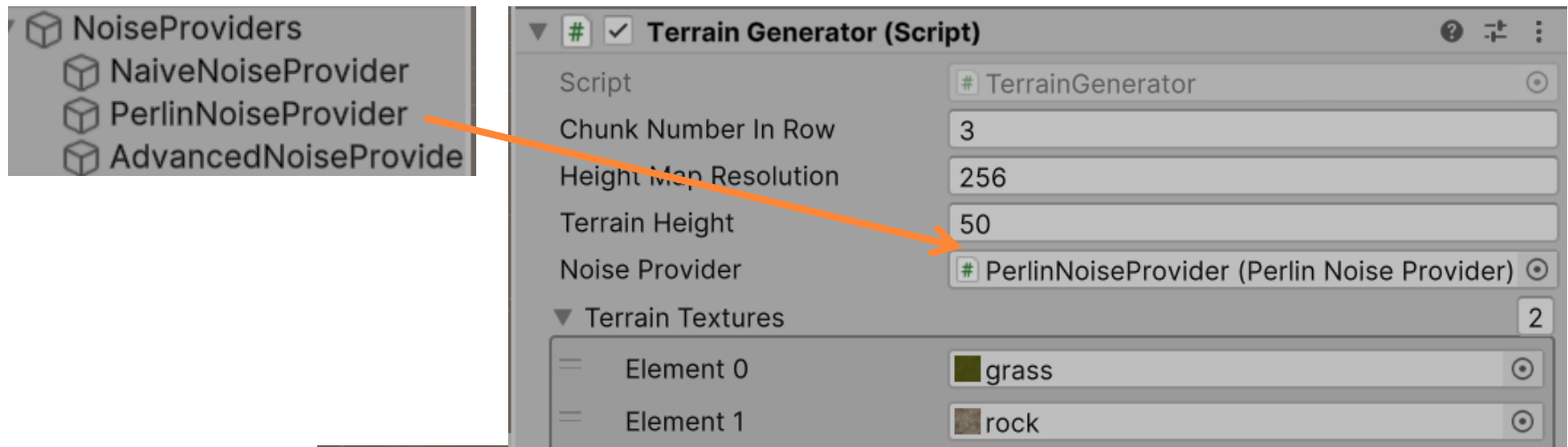
public class BaseNoiseProvider : MonoBehaviour, INoiseProvider
{
    virtual public float GetValue(float x, float z)
    {
        throw new NotImplementedException();
    }
}

class PerlinNoiseProvider : BaseNoiseProvider, INoiseProvider
{
    // LibNoise.Generator
    private static Perlin PerlinNoiseGenerator = new Perlin();

    override public float GetValue(float x, float z)
    {
        return (float)(PerlinNoiseGenerator.GetValue(x, 0, z) / 2f) + 0.5f;
    }
}
```



PRZYKŁAD GENERACJI TERENU, C.D.



PRZYKŁAD GENERACJI TERENU, C.D.

○ Tworzenie terenu

```
public void CreateTerrain(Texture2D[] textures)
{
    var terrainData = PrepareTerrainData();
    CreateTerrainLayers(terrainData, textures);
    ApplyTextures(terrainData, textures);

    var newTerrain = Terrain.CreateTerrainGameObject(terrainData);
    newTerrain.transform.position =
        new Vector3(Position.x * TerrainWidth, 0, Position.z * TerrainWidth);
    Terrain = newTerrain.GetComponent<Terrain>();
    Terrain.Flush();
}

private TerrainData PrepareTerrainData() {
    var terrainData = new TerrainData();
    terrainData.heightmapResolution = HeightMapResolution;
    terrainData.alphamapResolution = HeightMapResolution - 1;
    terrainData.size = new Vector3(TerrainWidth, TerrainHeight, TerrainWidth);

    terrainData.SetHeights(0, 0, GetHeightmap());
    return terrainData;
}
```

PRZYKŁAD GENERACJI TERENU, C.D.

○ Tworzenie warstw

```
private void CreateTerrainLayers(TerrainData terrainData, Texture2D[] textures)
{
    TerrainLayer[] terrainLayers = new TerrainLayer[textures.Length];

    for (int i = 0; i < textures.Length; i++)
    {
        terrainLayers[i] = new TerrainLayer();
        terrainLayers[i].diffuseTexture = textures[i];
    }

    terrainData.terrainLayers = terrainLayers;
}
```



PRZYKŁAD GENERACJI TERENU, C.D.

○ Tekstutowanie terenu

```
private void ApplyTextures(TerrainData terrainData, Texture2D[] textures)
{
    float[, ,] alphas =
        terrainData.GetAlphas(0, 0, terrainData.alphasWidth, terrainData.alphasHeight);
    int numLayers = terrainData.terrainLayers.Length;

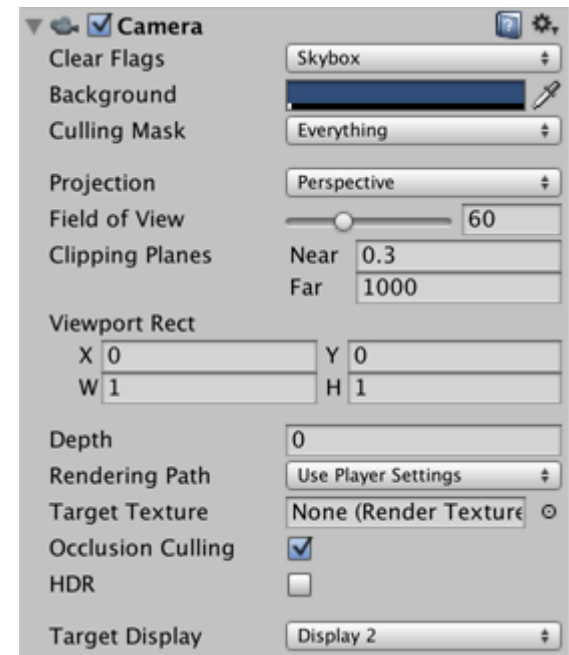
    for (var zRes = 0; zRes < terrainData.alphasHeight; zRes++)
    {
        for (var xRes = 0; xRes < terrainData.alphasWidth; xRes++)
        {
            for (int i = 0; i < numLayers; i++)
                alphas[zRes, xRes, i] = (i < textures.Length) ? 1f / numLayers : 0f;
        }
    }

    terrainData.SetAlphas(0, 0, alphas);
}
```



KAMERA

- Najważniejsze parametry:
 - Clear Flags – na ile stan poprzedniej ramki wpływa na bieżącą
 - Skybox, Solid Color – brak wpływu; nowa ramka rysowana na ustalonym tle
 - Depth only – kasowana informacja o odległości obiektu, kolor zachowany (zwykle z innej kamery)
 - Don't clear – nakładanie obrazów
 - Field of View – odpowiednik zoom
 - Viewpoint Rect – definicja obszaru na ekranie, na którym renderowany jest obraz z kamery
 - Depth – kolejność renderowania kamer (wyższe numery później)



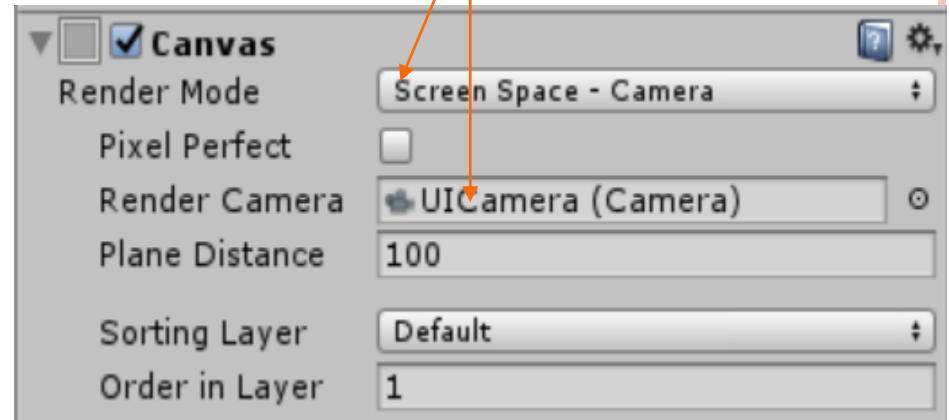
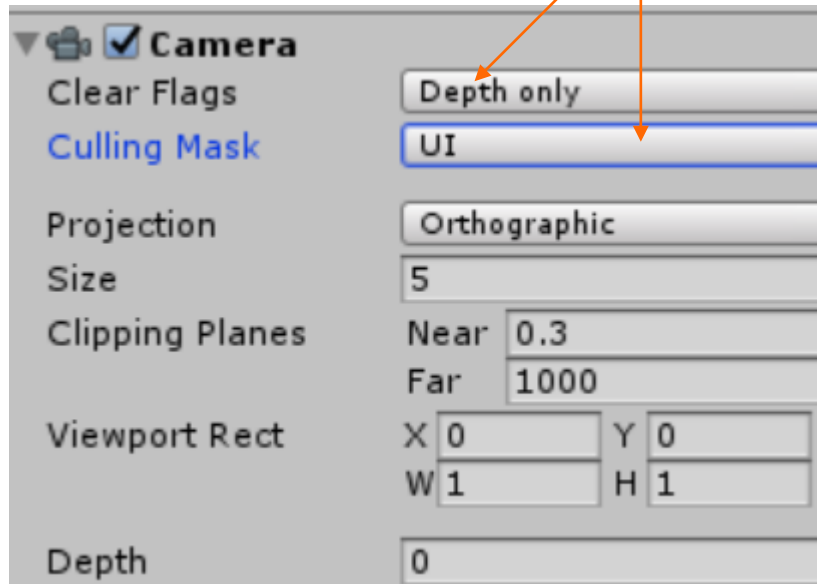
KAMERA, C.D.

- Kamery są zwykłymi obiektami gry – mogą być instantowane, animowane, sterowane skryptami, mieć dowiązany komponent Rigidbody
- Można mieć dowolną liczbę kamer na scenie
- W grach typu wyścigi można zwiększyć pole widzenia (Field of View) w celu poprawienia poczucia prędkości
- Zaleca się stosowanie kamer ortograficznych do wyświetlania obiektów z warstwy UI



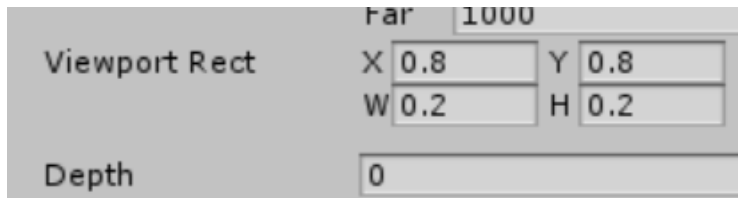
WYKORZYSTANIE WIELU KAMER, C.D.

- Wykorzystanie warstw do wyświetlania obiektów UI

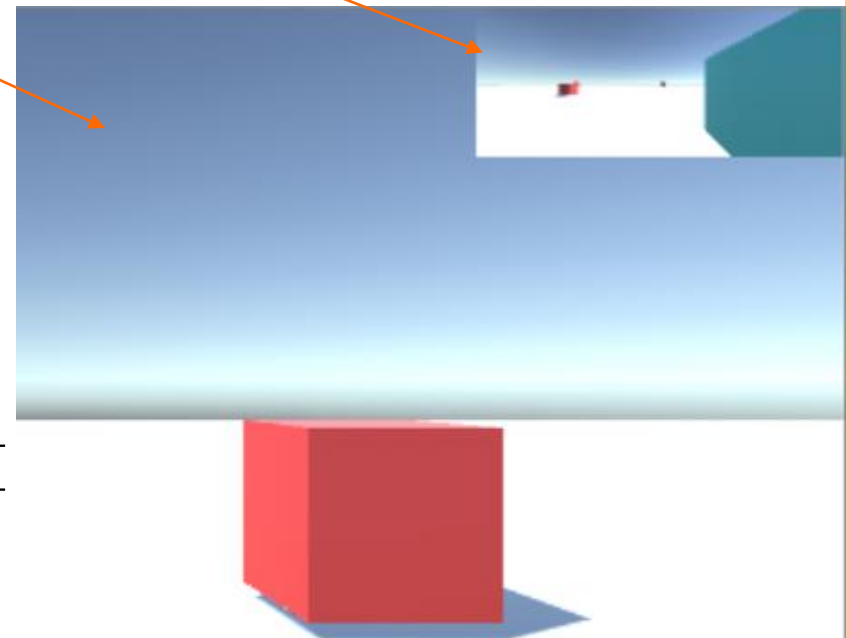


WYKORZYSTANIE WIELU KAMER

- Samochód ma dowiązane 3 kamery
- Wyświetlane są (ustawienia gracza):
 - MainCamera + MirrorCamera lub
 - TopCamera + MirrorCamera
- Ustawienia MirrorCamera:



- Na życzenie wyświetlana jest kamera na warstwie UI (UICamera) z dodatkowymi informacjami



WYKORZYSTANIE WIELU KAMER, C.D.

○ Przełączanie kamer

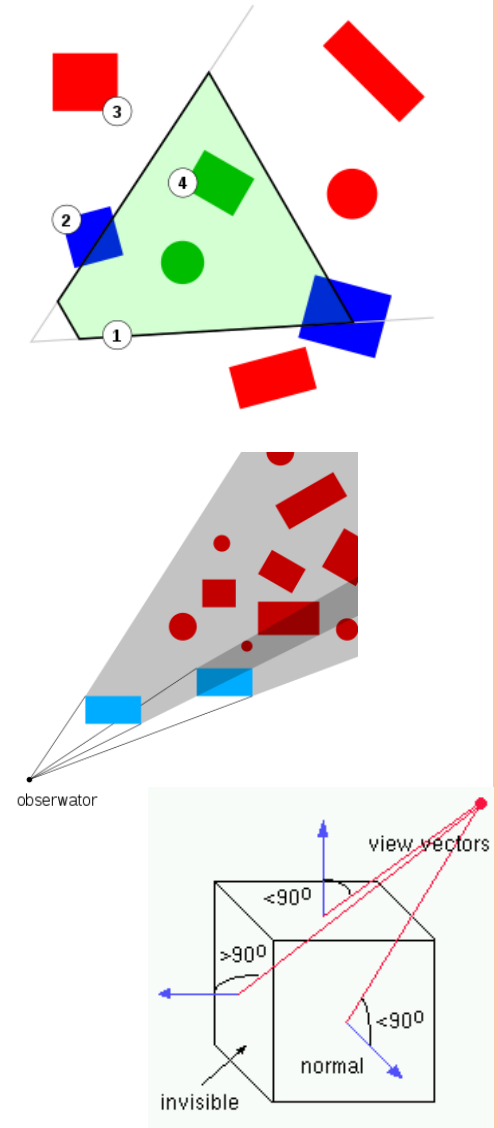
```
private GameObject mainCamera;  
private GameObject topCamera;  
private GameObject uiCamera;  
  
void Start () {  
    mainCamera = GameObject.Find("/Car/MainCamera");  
    topCamera = GameObject.Find("/Car/TopCamera");  
    uiCamera = GameObject.Find("UICamera");  
    topCamera.SetActive(false);  
    uiCamera.SetActive(false);  
}
```

```
void Update () {  
    // Sterowanie samochodem  
  
    if (Input.GetKeyDown(KeyCode.T)) {  
        mainCamera.SetActive(false);  
        topCamera.SetActive(true);  
    }  
    else if (Input.GetKeyDown(KeyCode.M)) {  
        mainCamera.SetActive(true);  
        topCamera.SetActive(false);  
    }  
    else if (Input.GetKeyDown(KeyCode.I))  
    {  
        if (uiCamera.activeSelf)  
            uiCamera.SetActive(false);  
        else  
            uiCamera.SetActive(true);  
    }  
}
```

USUWANIE NIEWIDOCZNYCH POWIERZCHNI (CULLING)

- Odrzucanie obiektów poza bryłą widzenia (Frustum culling) – automatycznie realizowane w Unity
- Odrzucanie obiektów, które są zasłanianie przez inne obiekty bliżej od obserwatora (Occlusion culling) – ten element trzeba włączyć na życzenie
- Odrzucanie tylnych ścian (Backface culling) – nie są wyświetlane ściany wielokątów „od wewnątrz”

https://pl.wikipedia.org/wiki/Usuwanie_niewidocznych_powierzchni

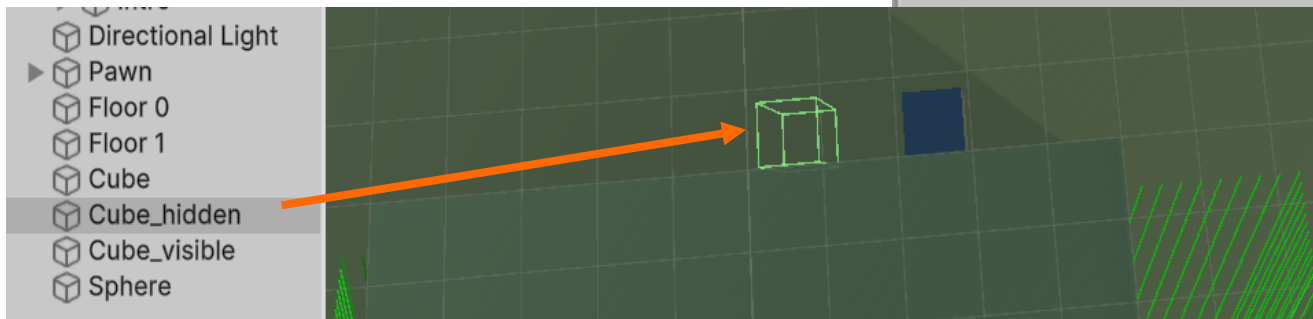
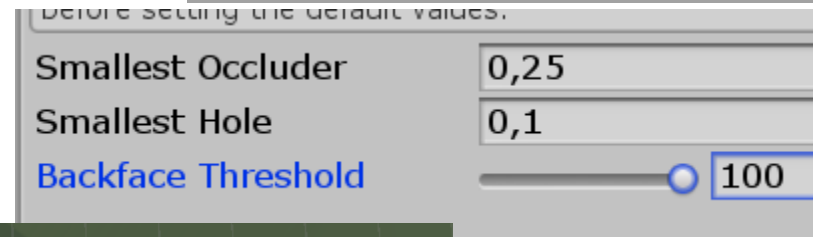
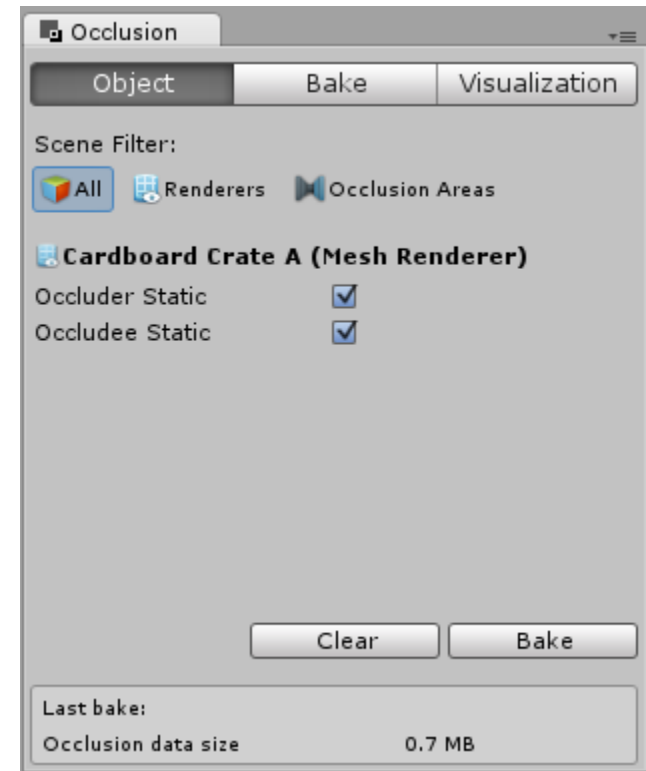


USUWANIE NIEWIDOCZNYCH POWIERZCHNI

- Window > Rendering > Occlusion Culling
- Obiekty statyczne dodatkowo się oznacza:

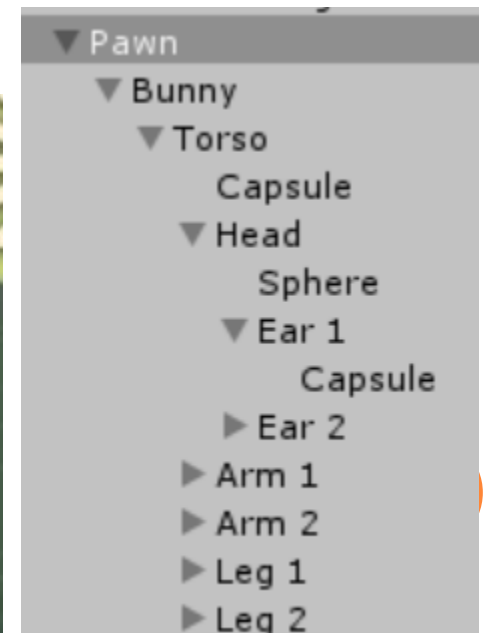
 - Occluder Static – obiekt, który zasłania (musi być oznaczony)
 - Ocludee Static – obiekty, które jest zasłaniany (sam nie zasłania)

- Wyniki obliczeń okluzji należy zapisać (Bake)



PRZYKŁADOWA GRA 3D

- Postać – królik zbudowany z kapsuł, połączonych złączami (sprężynami); bryła sztywna, która nie rotuje po osiach.
- Zastosowane elementy:
 - Mapa światła
 - Składanie obrotów w Unity
 - Animacje – drzewa mieszające
 - Obiekty skryptowe
 - Ragdoll



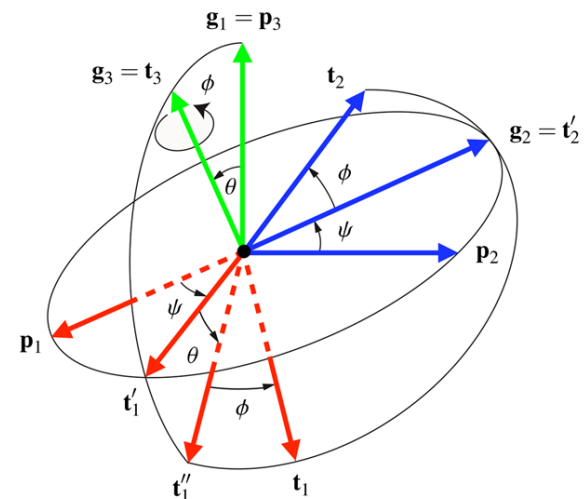
OBROTY W UNITY

- Kwaternion (Quaternion) – reprezentacja obrotu (wykorzystywana wewnętrznie w Unity), na którą składa się wektor (x, y, z) oraz skalar (w); Unity automatycznie przelicza tę reprezentację na kąty Eulera
- Najczęściej wykorzystywane funkcje:
 - Quaternion.Euler – metoda fabrykująca
 - Quaternion.Slerp – sferyczna interpolacja między dwoma punktami z krokiem t
 - Quaternion.Identity – brak rotacji
- $Q1 * Q2$ – składanie obrotów
- $Q * \text{wektor}$ – rotacja wektora, np.

```
var rot = Quaternion.Euler(0, 45, 0);
```

```
var v45 = rot * Vector3.forward;
```

```
// rotuje wektor „forward” o 45 stopni wokół osi Y
```



OBROTY W UNITY, PRZYKŁADOWA GRA 3D

- Kamera zawsze za graczem; gdy gracz się obraca, kamera zmienia położenie (offset) oraz obrót (rotation)

```
void UpdateView()
{
    var xRot = Quaternion.Euler(Mathf.Clamp(-_lookAngle.y, 0f, 90f), 0f, 0f);
    var yRot = Quaternion.Euler(0f, _lookAngle.x, 0f);

    var offsetRotX = xRot * AttachOffset;
    var offsetRotXY = yRot * offsetRotX;

    transform.position = _followPositon + offsetRotXY;
    transform.rotation = yRot * xRot;
}
```



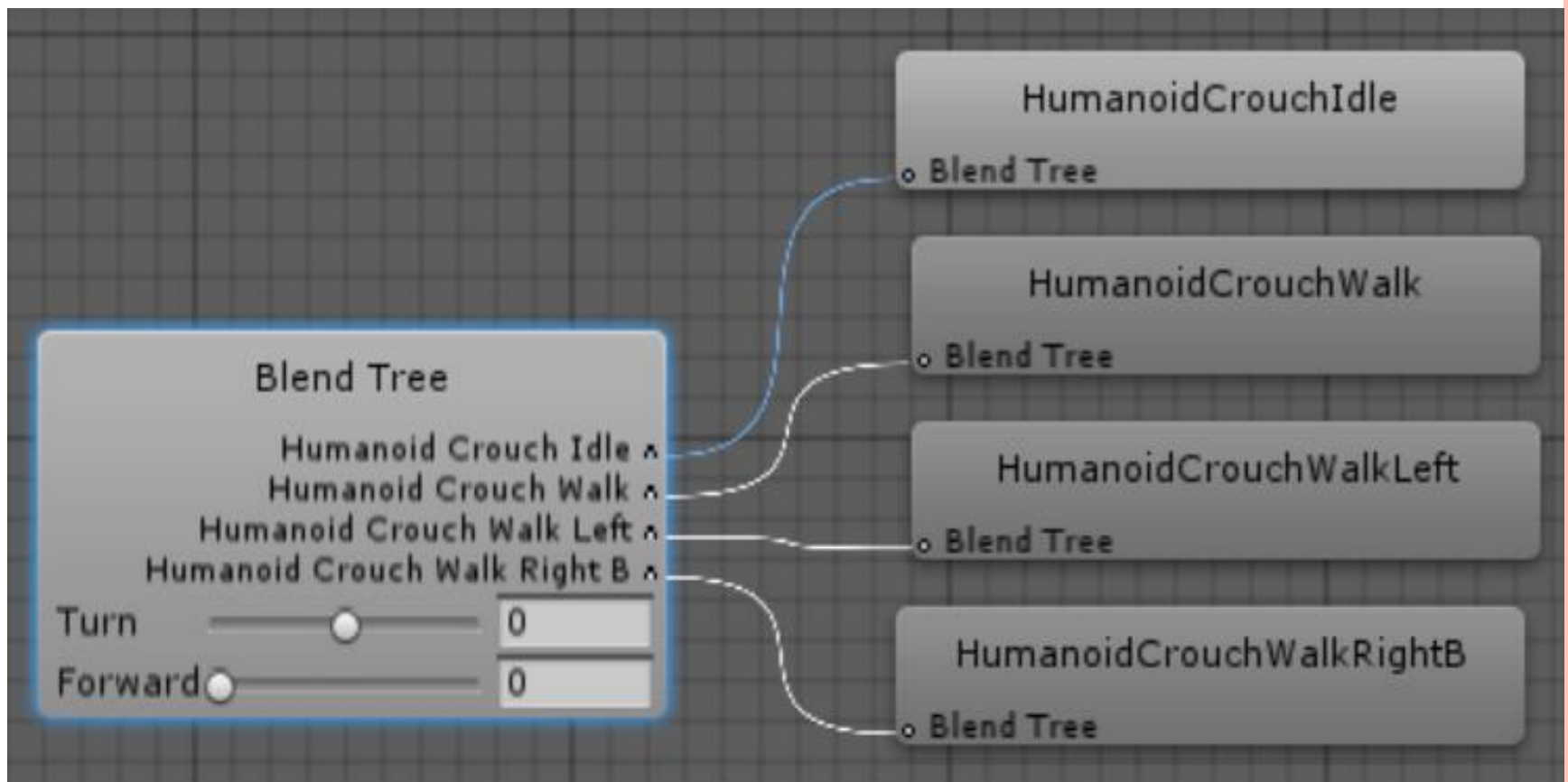
ANIMACJE – DRZEWA MIESZAJĄCE (BLEND TREES)

- Maszyna stanów opisująca animację może mieć specjalne węzły, zwane drzewami mieszającymi
- Zadaniem drzewa mieszającego jest łączenie dwóch lub więcej animacji w zależności od stanu gry, np. połączenie chodzenia i biegu w zależności od prędkości postaci
- Typy:
 - 1D – na podstawie pojedynczego parametru
 - 2D – na podstawie dwóch parametrów
 - Direct – dowolna liczba parametrów; każdy steruje wagą jednego klipu



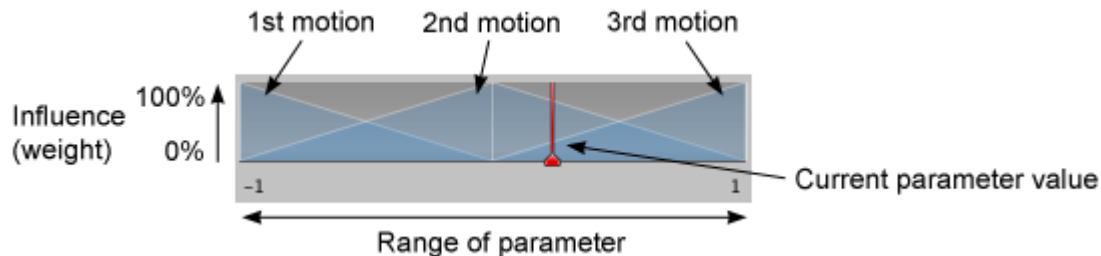
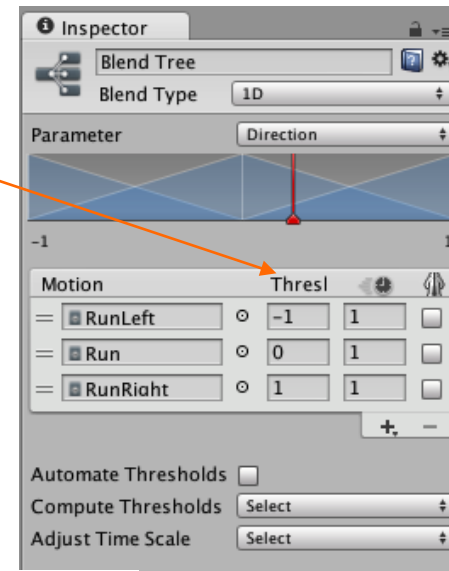
ANIMACJE – DRZEWA MIESZAJĄCE (BLEND TREES)

- Przykład drzewa mieszającego



BLENDOWANIE 1D

- Wartości progowe parametru
- Interpretacja wartości progowych



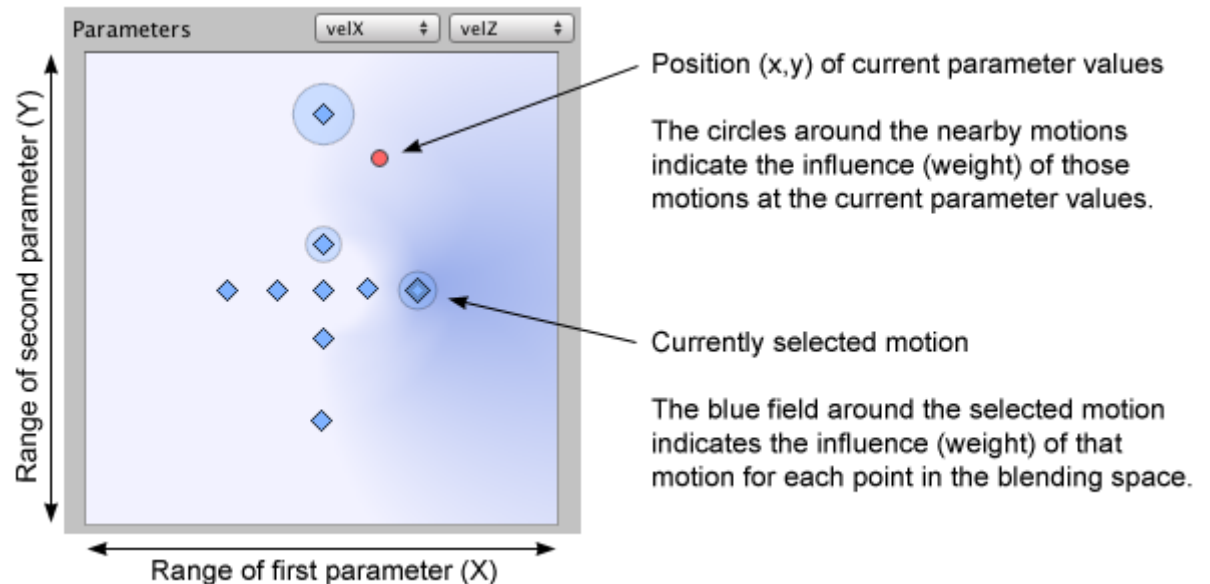
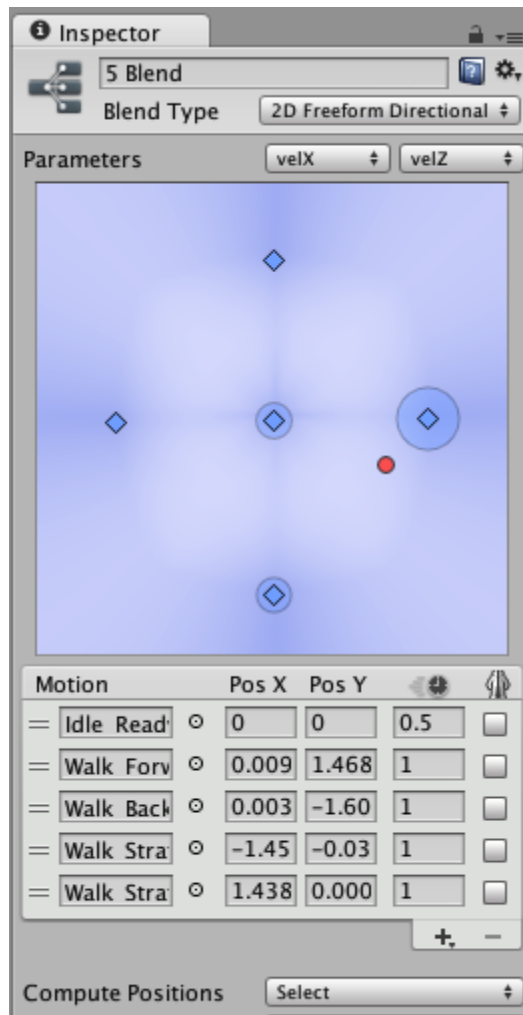
BLENDOWANIE 2D

- Typy (przykładowe):
 - 2D Simple Directional – ruchy w różnych kierunkach, nie ma wielu ruchów w tym samym kierunku
 - 2D Freeform Directional – ruchy w różnych kierunkach lub w tym samym kierunku z osobnym ruchem dla parametrów (0,0)
 - 2D Freeform Cartesian – ruchy w tym samym kierunku, np. blendowanie różnych animacji stanu Idle



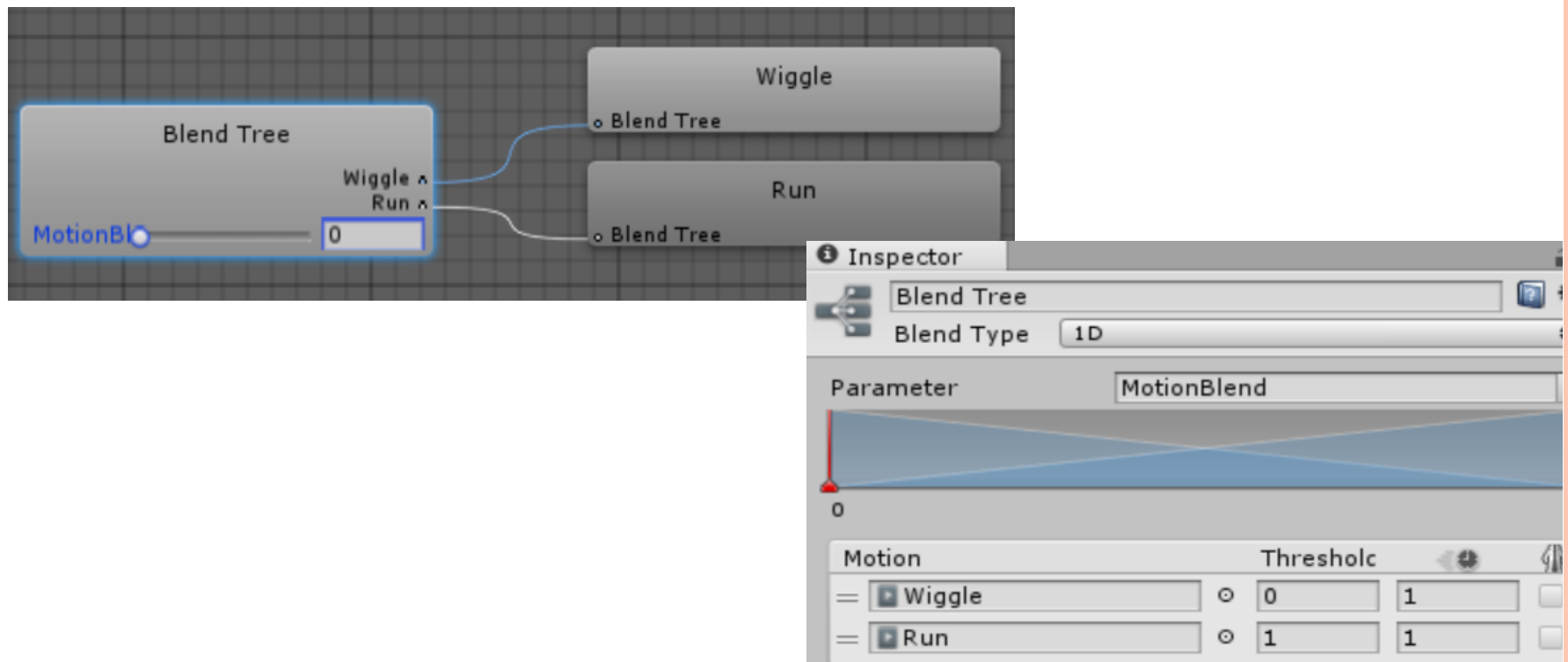
BLENDOWANIE 2D, C.D.

Definicja wartości parametrów i ich interpretacja



DRZEWA MIESZAJĄCE, PRZYKŁADOWA GRA 3D

- Animacja postaci wykorzystuje drzewa mieszające. Mieszane są dwa klipy: Wiggle (kołysanie na boki) oraz Run (bieg)
- Typ: 1D, Parametr: MotionBlend



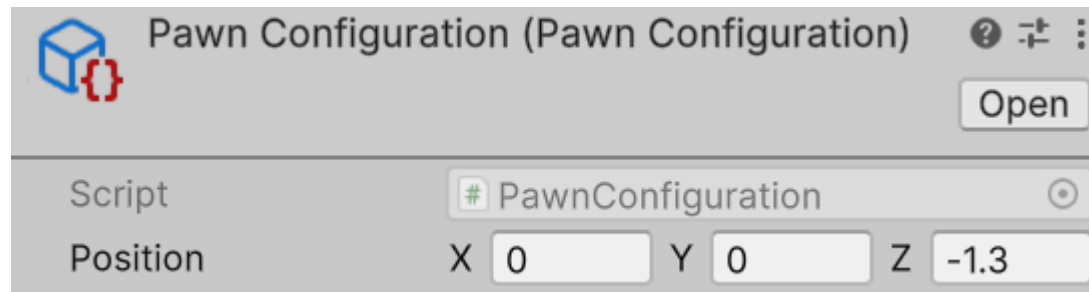
OBIEKTY SKRYPTOWE (SCRIPTABLE OBJECTS)

- Problemy z MonoBehaviour:
 - Niska wydajność (mnóstwo funkcji callbackowych z Unity)
 - Problemy z serializacją (brak wsparcia m.in. dla polimorfizmu; zmiany w Inspektorze w trakcie gry nie są utrwalane)
 - Problemy z dzieleniem między scenami/projektami (konieczność przygotowania prefabów)
 - Niezbyt nadaje się do przechowywania ustawień (niektóre komponenty zbędne, np. Transform)



SCRIPTABLE OBJECTS, C.D.

- ScriptableObject – Klasa istniejąca równolegle do MonoBehaviour
- Służy do przechowywania danych/konfiguracji – kontener danych
- Nie jest komponentem, nie może być dołączana do GameObject
- Rzadko wywoływana przez Unity (OnEnable(), OnDisable(), OnDestroy())
- Jej instancja może być zapisana jako plik assetów (.asset)



SCRIPTABLE OBJECTS, PRZYKŁADOWA GRA 3D

- Atrybut `CreateAssetMenu` – atrybut stosowany do klas pochodnych od `ScriptableObject`; tworzy podmenu w edytorze w ramach opcji `Assets/Create`, pozwalający utworzyć zasób (asset) typu `ScriptableObject`
 - `fileName` – domyślna nazwa pliku dla nowych instancji
 - `menuName` – nazwa opcji w podmenu
 - `order` – pozycja elementu w menu

```
[CreateAssetMenu(fileName = "PawnConfiguration",  
                  menuName = "Scriptable Objects/PawnConfiguration")]
```

Skrypt aparatu Unity | 1 odwołanie

```
public class PawnConfiguration : ScriptableObject  
{  
    public Vector3 position;  
}
```



SCRIPTABLE OBJECTS, PRZYKŁADOWA GRA 3D

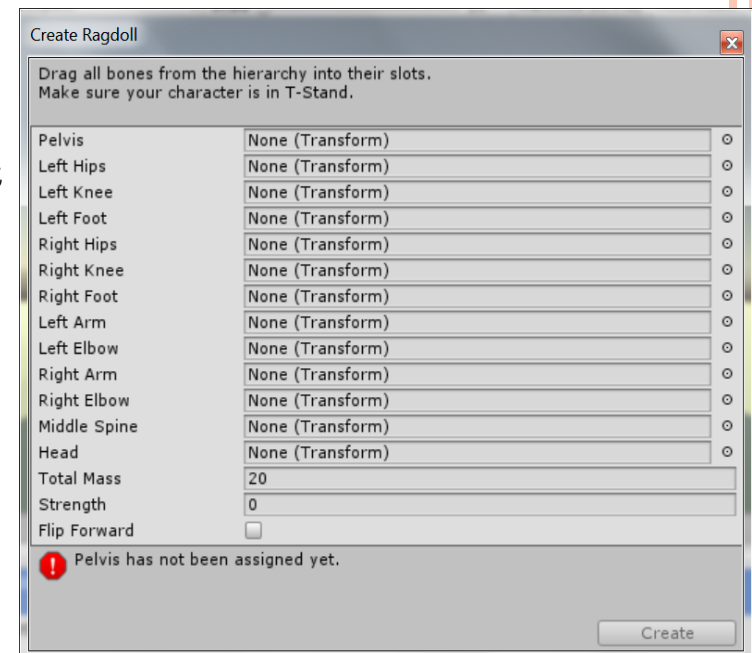
- Wykorzystanie obiektu z danymi

```
public PawnConfiguration pawnPosition;  
📦 Unity Message | Odwołania: 2  
protected virtual void Awake()  
{  
    Rigidbody = GetComponent<Rigidbody>();  
    transform.rotation = Quaternion.identity;  
    Rigidbody.MovePosition(pawnPosition.position);  
}
```



RAGDOLL – UMIERANIE POSTACI

- Dla postaci, która jest ciałem kinematycznym, animowanym przez skrypt:
 - Wyłącz animator
 - Dla każdej kończyny:
 - Wyłącz isKinematic
 - Włącz zderzacze
 - Włącz grawitację
 - Nadaj kończynom jakąś prędkość
- Dla postaci zbudowanej w standardowy sposób (mapowanie do podstawowych kości), można skorzystać z kreatora (GameObject > 3D Object > Ragdoll) – automatycznie wygeneruje zderzacze, komponenty Rigidbody i złącza



RAGDOLL, PRZYKŁADOWA GRA 3D

- Umieranie postaci - skrypt AnimatedPawn

```
public override bool Kill()
{
    var velocity = Rigidbody.velocity;

    if (base.Kill())
    {
        Animator.enabled = false;

        foreach (var limb in _limbs)
        {
            limb.GetComponent<Collider>().enabled = true;
            limb.isKinematic = false;
            limb.useGravity = true;
            limb.interpolation = RigidbodyInterpolation.Interpolate;
            limb.velocity = velocity;
        }

        return true;
    }

    return false;
}
```



PYTANIA KONTROLNE

- Jakie typowe zadania można wykonać w trakcie modelowania terenu?
- Wymień przykładowe szumy, które mogą być wykorzystane przy generacji terenu i biomów
- Wymień najważniejsze parametry kamery
- Na czym polega Occlusion Culling?
- Do czego służą drzewa mieszające? Jakie typy drzew mieszających można zdefiniować w Unity
- Do czego w Unity służy kwaternion? Jakie operacje najczęściej na nim się wykonuje?
- Czym jest Ragdoll?
- Do czego służy obiekt skryptowy? Jakim podlega ograniczeniom?

